Here is a comprehensive list of commonly asked **Data Structure Programming Questions** with **Java 1.8 code examples** that are often seen in interviews:

Array-Based Questions

1. Find the Maximum and Minimum in an Array

```
int[] arr = {10, 20, 5, 25, 15};
int max = Arrays.stream(arr).max().getAsInt();
int min = Arrays.stream(arr).min().getAsInt();
System.out.println("Max: " + max + ", Min: " + min);
```

2. Reverse an Array

```
int[] arr = {1, 2, 3, 4, 5};
for (int i = 0, j = arr.length - 1; i < j; i++, j--) {
   int temp = arr[i];
   arr[i] = arr[j];
   arr[j] = temp;
}
System.out.println(Arrays.toString(arr));
```

3. Remove Duplicates from Array

```
int[] arr = {1, 2, 2, 3, 4, 4};
int[] unique = Arrays.stream(arr).distinct().toArray();
System.out.println(Arrays.toString(unique));
```

String-Based Questions

4. Check if a String is a Palindrome

```
String s = "madam";
boolean isPalindrome = new StringBuilder(s).reverse().toString().equals(s);
System.out.println("Palindrome: " + isPalindrome);
```

5. Count Characters in a String

```
String input = "interview";
Map<Character, Long> freq = input.chars()
   .mapToObj(c -> (char)c)
   .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
System.out.println(freq);
```

Linked List

6. Reverse a Linked List

```
class Node {
   int data;
   Node next;
   Node(int d) { data = d; }
}
Node reverse(Node head) {
   Node prev = null, current = head, next;
   while (current != null) {
      next = current.next;
      current.next = prev;
      prev = current;
      current = next;
   }
   return prev;
}
```

7. Detect a Cycle in Linked List (Floyd's Algorithm)

```
boolean hasCycle(Node head) {
   Node slow = head, fast = head;
   while (fast != null && fast.next != null) {
      slow = slow.next;
      fast = fast.next.next;
      if (slow == fast) return true;
   }
   return false;
}
```

Stack & Queue

8. Implement Stack Using Array

```
class Stack {
   int top = -1, max = 1000;
   int[] stack = new int[max];

   void push(int x) {
      if (top == max - 1) return;
      stack[++top] = x;
   }

   int pop() {
      if (top < 0) return -1;
      return stack[top--];
   }
}
```

9. Balanced Parentheses using Stack

```
boolean isBalanced(String expr) {
   Stack<Character> stack = new Stack<>();
   for (char c : expr.toCharArray()) {
      if (c == '(' || c == '{' || c == '[') stack.push(c);
      else if (c == ')' && (stack.isEmpty() || stack.pop() != '(')) return false;
      else if (c == '}' && (stack.isEmpty() || stack.pop() != '{')) return false;
      else if (c == ']' && (stack.isEmpty() || stack.pop() != '[')) return false;
   }
   return stack.isEmpty();
}
```

Tree-Based

10. In-Order Traversal (Recursive)

```
void inorder(TreeNode root) {
   if (root != null) {
      inorder(root.left);
      System.out.print(root.val + " ");
      inorder(root.right);
   }
}
```

11. Level-Order Traversal

```
void levelOrder(TreeNode root) {
   Queue<TreeNode> q = new LinkedList<>();
   q.add(root);
   while (!q.isEmpty()) {
      TreeNode node = q.poll();
      System.out.print(node.val + " ");
      if (node.left != null) q.add(node.left);
      if (node.right != null) q.add(node.right);
   }
}
```

HashMap / HashSet

12. Find First Non-Repeating Character

```
String s = "interview";
Map<Character, Integer> map = new LinkedHashMap<>();
for (char c : s.toCharArray())
   map.put(c, map.getOrDefault(c, 0) + 1);
```

```
for (Map.Entry<Character, Integer> e : map.entrySet()) {
   if (e.getValue() == 1) {
      System.out.println("First non-repeating: " + e.getKey());
      break;
   }
}
```

13. Two Sum Problem (Pair with Given Sum)

```
int[] arr = {2, 7, 11, 15};
int target = 9;
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
   if (map.containsKey(target - arr[i])) {
      System.out.println("Pair: " + map.get(target - arr[i]) + ", " + i);
   }
   map.put(arr[i], i);
}
```

Sorting and Searching

14. Binary Search (Iterative)

```
int binarySearch(int[] arr, int key) {
   int low = 0, high = arr.length - 1;
   while (low <= high) {
      int mid = (low + high) / 2;
      if (arr[mid] == key) return mid;
      else if (arr[mid] < key) low = mid + 1;
      else high = mid - 1;
   }
   return -1;
}
```

15. Quick Sort

```
void quickSort(int[] arr, int low, int high) {
   if (low < high) {
      int pi = partition(arr, low, high);
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);
   }
}
int partition(int[] arr, int low, int high) {
   int pivot = arr[high], i = (low - 1);
   for (int j = low; j < high; j++) {
      if (arr[j] < pivot) {
         i++;
```

```
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
      }
    }
    int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
    return i + 1;
}
```

Would you like this entire set as a **PDF**, **Excel checklist**, or continue with **graph algorithms**, **heap**, or **real-time DSA-based scenarios**?

Let me know and I'll generate the next part!

Great! Let's now expand the **Java 1.8 Data Structure Interview Set** to include:

1. **Graph Algorithms**
2. **Heap/Priority Queue Use Cases**
3. **Real-Time DSA-Based Scenarios**

---

1. Graph Algorithms in Java

a. Breadth-First Search (BFS)

```
void bfs(int start, Map<Integer, List<Integer>> graph) {
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();
    queue.add(start);
    visited.add(start);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.print(node + " ");
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.add(neighbor);
            }
        }
    }
}
```

b. Depth-First Search (DFS)

```
void dfs(int node, Map<Integer, List<Integer>> graph, Set<Integer> visited) {
    visited.add(node);
    System.out.print(node + " ");
    for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
```

```
        if (!visited.contains(neighbor)) {
            dfs(neighbor, graph, visited);
        }
    }
}
```

c. Detect Cycle in Undirected Graph (Union-Find)

```
int find(int[] parent, int i) {
    if (parent[i] == -1) return i;
    return find(parent, parent[i]);
}
boolean union(int[] parent, int x, int y) {
    int xset = find(parent, x);
    int yset = find(parent, y);
    if (xset == yset) return true;
    parent[xset] = yset;
    return false;
}
```

---

2. Heap / PriorityQueue Examples

a. Find K Largest Elements

```
int[] arr = {3, 1, 5, 12, 2, 11};
int k = 3;
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
for (int num : arr) {
    minHeap.add(num);
    if (minHeap.size() > k) minHeap.poll();
}
System.out.println("Top K elements: " + minHeap);
```

b. Sort a Nearly Sorted (K-sorted) Array

```
int[] arr = {2, 6, 3, 12, 56, 8};
int k = 3;
PriorityQueue<Integer> heap = new PriorityQueue<>();
int index = 0;
for (int i = 0; i < arr.length; i++) {
    heap.add(arr[i]);
    if (heap.size() > k) arr[index++] = heap.poll();
}
while (!heap.isEmpty()) arr[index++] = heap.poll();
System.out.println(Arrays.toString(arr));
```

---

3. Real-Time DSA-Based Scenarios

a. LRU Cache using LinkedHashMap

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
   private final int capacity;
   public LRUCache(int capacity) {
      super(capacity, 0.75f, true);
      this.capacity = capacity;
   }
   protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
      return size() > capacity;
   }
}
```

b. Least Frequent Element Removal (LFU Logic)

```
Map<Integer, Integer> freq = new HashMap<>();
for (int n : arr) freq.put(n, freq.getOrDefault(n, 0) + 1);

int leastFreq = Collections.min(freq.values());
for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
   if (e.getValue() == leastFreq) {
      System.out.println("Remove: " + e.getKey());
      break;
   }
}
```

c. Autocomplete Suggestions using Trie

```
class TrieNode {
   Map<Character, TrieNode> children = new HashMap<>();
   boolean isWord = false;
}
class Trie {
   TrieNode root = new TrieNode();
   void insert(String word) {
      TrieNode node = root;
      for (char c : word.toCharArray())
         node = node.children.computeIfAbsent(c, k -> new TrieNode());
      node.isWord = true;
   }

   List<String> autocomplete(String prefix) {
      List<String> results = new ArrayList<>();
      TrieNode node = root;
      for (char c : prefix.toCharArray())
         node = node.children.get(c);
      dfs(node, new StringBuilder(prefix), results);
      return results;
   }
```

```
    void dfs(TrieNode node, StringBuilder prefix, List<String> results) {
        if (node == null) return;
        if (node.isWord) results.add(prefix.toString());
        for (char c : node.children.keySet()) {
            dfs(node.children.get(c), prefix.append(c), results);
            prefix.deleteCharAt(prefix.length() - 1);
        }
    }
}
```

What's Next?

Would you like the following in the next list?

- **Sliding Window**, **Two Pointer**, **Greedy Algorithms**
- **Dynamic Programming (DP)**
- **Concurrency / Multi-threading related DSA use cases**
- **Convert all of this into a printable PDF or Notion-style checklist**

Let me know your preferred format or next topic!